

DeepSeek Harness 二次开发与版本同步指南

vibe coding 实战手册

2026-08-14

摘要

本文面向想要基于 DeepSeek Harness (dsh, 当前版本 0.1.0-rc.5) 做二次开发、同时在官方不断升级版本时能把自有改动平滑迁移到新版本的开发者。开发以二次开发为主、官方更新时同步跟上。文档从 git 零基础讲起, 先厘清“源码从哪来、如何安装构建”, 再对比“改核心包”与“写插件”两条路线, 最后给出以 git 分支与 vibe coding 为核心、可反复执行的升级同步流程。推荐的托管形态是 **Fork + 双远程**: Fork 提供你账号下的云端备份, **upstream** 远程负责跟随官方更新。文中的命令均在实际开发仓库 (penguin 的 fork 克隆) 上核对。

目录

1	背景与总体思路	2
1.1	这个仓库是什么	2
1.2	当前这份文件夹的状态	3
1.3	二次开发的目标	3
1.4	核心结论 (先给答案)	3
2	第一步: 建立 Fork + 双远程的仓库结构	3
2.1	方式 A (推荐): Fork + 克隆 + 关联 upstream	3
2.2	方式 B (备选): 对现有 ZIP 快照就地 git init	4
3	第二步: 安装依赖与构建	4
3.1	环境要求	4
3.2	安装与验证	5
3.3	日常启动: 一键运行 + API 配置 (本机已配置)	5
3.4	前端增强: dsh-web-ui 插件全家桶 (本机已安装)	5
4	git 零基础入门 (够用的那部分)	6
4.1	核心概念	6
4.2	每日必用命令	6

1 背景与总体思路	2
4.3 从改代码到提交：一个完整示例	7
4.4 常见的“我该用哪个命令”对照	8
4.5 最小安全工作流	8
4.6 必懂的一个概念：合并（merge）与变基（rebase）	8
5 两条二次开发路线	9
5.1 路线 A：直接修改核心包（不推荐用于长期定制）	9
5.2 路线 B：写插件（推荐）	9
5.3 路线对比	10
6 升级同步流程（核心）	10
6.1 策略总览	10
6.2 升级操作的详细步骤	11
6.3 冲突是什么、怎么处理	11
6.4 把升级与冲突交给 vibe coding	12
6.5 写给 AI 的提示词模板	12
6.6 长期维护建议	12
7 故障排查（实战记录）	13
7.1 双击启动脚本闪退	13
7.2 Web 启动报 <code>plugin tree failed to load</code>	13
7.3 皮肤中心「应用失败 ENOENT scandir node_modules/skins/」	14
7.4 技术文档位置错乱	14
7.5 皮肤缺款：鲸吟（whale-song）无法切换	15
7.6 外部皮肤：深海女仆工坊（maid-atelier）	15
7.7 pnpm store 版本冲突	15
7.8 故障排查通用流程	16
8 常见问题	16

1 背景与总体思路

1.1 这个仓库是什么

DeepSeek Harness 是 DeepSeek AI 开源的一个 agent harness（智能体运行框架），采用“万物皆插件（everything is a plugin）”的架构，底层是 vendored 的 Cordis 框架。当前版本 0.1.0-rc.5，处于开发者预览阶段，官方明确声明 会有破坏性变更（compatibility-breaking changes），并且 pre-release 策略是“优先正确的地基，而非兼容垫片”，这意味着官方随时可能重命名、重构、重打包内部 API。

1.2 当前这份文件夹的状态

最初你下载的 `deepseek-harness-master` 是从 GitHub 下载的 master 分支 ZIP 快照（无 `.git`）；本指南的正式开发仓库是 fork 克隆（如 `penguin` 目录，带完整 `git`）。两者的区别与注意点：

- **ZIP 快照（无 `.git`）**：没有提交历史、没有远程仓库关联、不能 `git pull/push/merge`。源码文件完整，`pnpm install`、`pnpm run typecheck`、`pnpm run build` 都不需要 `.git`（`Lefthook` 钩子检测不到 `git` 会静默跳过）。但二次开发会失去版本管理能力和升级同步能力，只适合做临时参考。
- **fork 克隆（有 `.git`）**：本指南默认的开发形态，具备完整 `git` 能力，见第 6 节。配置与文档只放在这一个仓库里（详见第 7.4 节）。

1.3 二次开发的目标

1. 在官方功能基础上做自己的定制（改行为、加工具、加界面、改提示词等）。
2. 当官方发布新版本时，能把自己做的改动快速、低冲突地“搬运”到新版本上。
3. 整个过程希望尽量借助 `vibe coding`（AI 辅助编程）自动化。

1.4 核心结论（先给答案）

1. 用 **Fork + 双远程托管**：在 GitHub 网页 Fork 官方仓库（得到你账号下的云端副本，天然备份），克隆自己的 Fork 到本地，并把官方仓库设为 `upstream` 远程。这是后面一切的地基。
2. 优先选择“写插件”路线，而不是改核心包：插件天然与上游解耦，官方升级时几乎不受影响，是最适合“低冲突升级”的形态。官方架构文档也把“新行为应放在扩展点上，而非改 `agent-loop`”列为纪律。
3. 用“分支 + 定期合入 `upstream`”管理升级：你的改动永远放在自己的分支上，官方升级通过 `git merge upstream/master` 合入，冲突时借助 `vibe coding` 辅助解决。

2 第一步：建立 Fork + 双远程的仓库结构

以二次开发为主、同时要跟上官方更新，最合适的形态是“Fork 一份 + 本地双远程”。这样你账号下有一份云端备份（不怕本地丢失），本地又能直接拉取官方更新。推荐方式 A。

2.1 方式 A（推荐）：Fork + 克隆 + 关联 upstream

第 1 步，Fork 官方仓库。用浏览器登录 GitHub，打开 <https://github.com/deepseek-ai/deepseek-harness>，点右上角 **Fork**，在你自己的账号下生成一份独立副本。Fork 是点击那一刻的快照，之后官方再更新不会自动同步过来（同步方法见第 6 节）。

第 2 步，克隆你自己的 Fork 到本地：

```
# 到一个干净的父目录下执行；替换为你的 GitHub 用户名
git clone https://github.com/<你的用户名>/deepseek-harness.git
cd deepseek-harness
```

第 3 步，把官方仓库设为 upstream 远程（origin 是你的 Fork，upstream 是官方）：

```
git remote add upstream https://github.com/deepseek-ai/deepseek-harness.git
git remote -v # 确认有两个远程：origin（你的 Fork）+ upstream（官方）
```

日常规律：

- 你改的代码 push 到 origin（自己的 Fork，云端备份）；
- 官方更新通过 git fetch upstream + 合并拉入（见第 6 节）；
- 想贡献回官方时，在 GitHub 上对你的 Fork 提 Pull Request，官方仓库会看到。

注意：拉下来后需重新 pnpm install（node_modules 不随 git 传递）。

2.2 方式 B（备选）：对现有 ZIP 快照就地 git init

如果你已经有这份 ZIP 快照目录、不想重新拉取（例如已安装好依赖），可以把当前目录初始化成仓库，并关联你自己的 Fork 与官方上游。先在网页 Fork 官方仓库，然后：

```
cd deepseek-harness-master # 现有快照目录
git init
git add -A
git commit -m "initial snapshot: dsh 0.1.0-rc.5 (GitHub master ZIP)"
git remote add origin https://github.com/<你的用户名>/deepseek-harness.git
git remote add upstream https://github.com/deepseek-ai/deepseek-harness.git
git fetch --all
git push -u origin master # 把快照推送到你的 Fork 备份
```

之后本地 master 就是你“存档快照”与官方最新代码的一个交汇点（会与官方提交历史发生冲突，因为 ZIP 快照不是由 git 提交产生的——这正是方式 A 更省事的原因）。若默认分支不是 master（官方可能改名 main），用 git branch -r 查看后以实际分支名替换。

3 第二步：安装依赖与构建

3.1 环境要求

- **Node.js**: ^22.19 || >=24（已核对：本机 v24.16.0 满足）。
- **pnpm**: 仓库钉死 pnpm@11.7.0（本机已满足），通过 Corepack 启用：corepack enable。
- **Git**: 2.26+（本机 2.47.0 满足）。

- 可选：DEEPSEEK_API_KEY（跑真实模型、Web UI 演示、e2e 测试时用，缺省时相关测试自动跳过）。

3.2 安装与验证

```
pnpm install      # 安装依赖; postinstall 会尝试装 lefthook (无 git 时自动跳过)
pnpm run typecheck # 官方规定: typecheck 通过才算安装完成
pnpm run build     # 构建 lib 与 Web 前端
pnpm dsh web       # 启动 Web UI, 默认 http://127.0.0.1:3080
```

若需要真实 API：在仓库根目录建 `.env`（已被 gitignore）：

```
DEEPSEEK_API_KEY=sk-xxx
# DEEPSEEK_BASE_URL=https://... # 可选, 默认官方公共接口
```

3.3 日常启动：一键运行 + API 配置（本机已配置）

本机（D:\soft\deepseek-harness-penguin\deepseek-harness）已配置好三件事，日常使用无需再手动敲命令：

1. 一键启动脚本：双击仓库根目录的 启动-DeepSeek-Harness.bat，即启动 Web UI 并打开浏览器（默认 `http://127.0.0.1:3080`）。
2. 桌面快捷方式：双击 创建桌面快捷方式.bat 生成 DeepSeek Harness.lnk 到桌面；右键该快捷方式 → 属性 → 快捷方式 → 更改图标，可自定义图标（需 `.ico/.exe/.dll`）。
3. `.env` 配置：API key 存于仓库根 `.env`（已被 gitignore，不提交）。主账号用 `DEEPSEEK_API_KEY`；多账号扩展预留字段 `DEEPSEEK_API_KEY_2` 等已注释在文件内，启用时在对应 profile 的 `cordis.yml` 里给 `llm-deepseek` 设 `apiKeyEnv: DEEPSEEK_API_KEY_2` 即可。

3.4 前端增强：dsh-web-ui 插件全家桶（本机已安装）

本机已在 web profile 安装社区插件 @linxin666/dsh-web-ui-all（来源：github.com/zhul090093659/dsh-web-ui，BSD-3-Clause），全部为插件形式、未改动核心代码，提供：

- 任务看板（支持 cron 定时执行）、Git 图谱、右侧文件/预览/SCM 面板；
- 实时 token 统计（TPS/耗时/缓存命中率）、鲸鱼娘宠物、移动端远程、SSH 远程运维；
- 皮肤中心：共 8 款皮肤，其中聚合包自带 6 款（Windows XP、Minecraft、蓝色幻想、QQ2008、同花顺、龙的传人），鲸吟（whale-song）需单独安装（见故障排查第 7.5 节），交易终端暂未随包分发。先试穿后应用。

安装/卸载命令（需要时可重装）：

```
dsh plugin --profile web add @linxin666/dsh-web-ui-all      # 安装全家桶
dsh plugin --profile web remove @linxin666/dsh-web-ui-all   # 卸载
# 单独装某个子插件，如仅皮肤：
dsh plugin --profile web add @linxin666/dsh-skins
```

Windows 注意：安装时若提示 `ERR_PNPM_IGNORED_BUILDS`，需在 `$DSH_HOME/profiles/web/pnpm-workspace.yaml` 的 `allowBuilds` 里把 `cloudflared` / `cpu-features` / `ssh2` 设为 `true`（SSH 与移动端公网隧道所需；本机已配置）。若 `ssh2` 因缺 VS C++ 工具链报 `gyp` 错误，属 `optional binding` 失败，不影响插件加载。

4 git 零基础入门（够用的那部分）

这一节只讲二次开发真正用得到的 git 概念与命令。遇到更复杂的场景再查官方文档或问 AI。

4.1 核心概念

- **工作区：**你正在编辑的文件夹。
- **暂存区 (index)：**git add 之后、git commit 之前的中间区。
- **提交 (commit)：**一个不可变的历史快照，带作者、时间、说明 (message)。
- **分支 (branch)：**一条独立的提交线。默认 master（或 main）。
- **远程 (remote)：**托管在网上的仓库。本指南的仓库有两个：`origin`（你的 Fork）和 `upstream`（官方仓库）。
- **HEAD：**当前所在提交的指针，通常指向某个分支。

4.2 每日必用命令

```
git status          # 看当前状态（改了什么、在哪个分支）
git diff            # 看未暂存的改动内容
git add <file>      # 把文件加入暂存区（git add -A 全部加入）
git commit -m "说明" # 提交
git log --oneline -10 # 看最近 10 条提交
git branch          # 列出本地分支，* 表示当前分支
git switch -c my-dev # 新建并切换到分支 my-dev（老写法：git checkout -b my-dev）
git switch master    # 切换回 master
git push -u origin my-dev # 首次把本地分支推到你的 Fork (origin) 并建立关联
git push            # 之后直接 push
git pull            # 拉取 origin/master 并与当前分支合并（fetch + merge）
git fetch origin     # 只拉取 origin（你的 Fork）信息，不合并
git fetch upstream   # 只拉取 upstream（官方）信息，不合并（更安全）
```

4.3 从改代码到提交：一个完整示例

下面用一条真实的工作流，把最常用的命令串起来看。重点看 `git commit` 的“更新说明”怎么写。

第 1 步：先建一个自己的开发分支，并切过去（不要直接在 `master` 上改）：

```
git switch -c feature-hello
# 输出: Switched to a new branch 'feature-hello'
```

第 2 步：用编辑器改代码（例如新建了 `scratch-plugin/src/my-plugin.ts`，并改了 `scratch-plugin/cordis.yml`）。

第 3 步：看一眼自己改了什么。`git status` 看“哪些文件变了”，`git diff` 看“具体改了什么”：

```
git status
# 输出示意:
# On branch feature-hello
# Untracked files:
#   scratch-plugin/cordis.yml
#   scratch-plugin/src/my-plugin.ts
# 没被 git 跟踪过的新文件
```

```
git diff
# 对已跟踪文件的改动显示 + / - 行；新文件默认不显示
```

第 4 步：把改动加入暂存区，然后提交。更新说明写在 `-m` 后面的引号里：

```
git add scratch-plugin/cordis.yml scratch-plugin/src/my-plugin.ts
# 或想省事: git add -A    # 把当前所有改动全部加入

git commit -m "feat: 新增 hello 演示插件, 可通过 --patch 挂载"
# 输出示意:
# [feature-hello <一串哈希>] feat: 新增 hello 演示插件, 可通过 --patch 挂载
# 2 files changed, 15 insertions(+)
```

说明：

- `-m` 后面的字符串就是这次提交的说明，将来你（或 AI）看历史时，靠它知道这次改了啥，所以写清楚“改了啥、为什么”比写“update”有用得多。
- 说明通常用英文或中文都可以，但建议带一个简短前缀，让人一眼看出类型，常见写法：

```
git commit -m "feat: 新增 hello 插件"      # feat = 新功能
git commit -m "fix: 修复日志乱码问题"      # fix = 修 bug
git commit -m "docs: 更新二次开发指南"     # docs = 改文档
git commit -m "refactor: 重构插件加载逻辑" # refactor = 重构, 行为不变
git commit -m "merge: 合并 upstream master 到 feature-hello" # 合并类
```

- 说明较长、想分多行写时，可以不带 `-m`，直接执行 `git commit`，git 会打开一个文本编辑器，

第一行写标题，空一行后写详细说明，保存退出即可。

第 5 步：查看提交历史，确认提交成功：

```
git log --oneline -5
# 输出示意（越往下越老）：
# b3f2a91 feat: 新增 hello 演示插件，可通过 --patch 挂载
# 9d0c41e docs: 更新二次开发指南
# ...
```

第 6 步：把分支推到你的 Fork 做云端备份：

```
git push -u origin feature-hello
# 首次加 -u 建立关联；以后在同一个分支上直接 git push 即可
```

4.4 常见的“我该用哪个命令”对照

我的想法	用什么命令
“我改了什么？”	git status（哪些文件）、git diff（具体行）
“这次改动想留个存档”	git add + git commit -m "说明"
“提交信息打错了，改一下”	git commit --amend -m "新说明"（只能改最近一次，且未推送时用）
“改到一半想先放着”	git stash（暂存）；git stash pop（取回）
“想撤销某个文件的改动”	git checkout -- <file>（回到最近一次提交的样子，小心：未提交的改动会丢失）
“想回到某个提交”	git reset --hard <哈希>（小心：会丢提交；用 git reflog 可找回）
“看远程有哪些分支/提交”	git fetch origin、git log origin/my-dev --oneline

上述“危险”命令建议先在测试仓库里练一遍再对正式仓库用；拿不准时先问 AI，把 git status 的输出贴给它。

4.5 最小安全工作流

- 1. 永远不在 master 上直接开发。每次新改动：git switch -c feature-xxx。
- 2. 改一点就提交一点：git add + git commit。提交信息写清楚“改了啥、为什么”。
- 3. 开发分支推送到你的 Fork 做云端备份：git push -u origin feature-xxx，避免本地丢失。
- 4. 想要官方更新时（见下节），在分支里合并 upstream。

4.6 必懂的一个概念：合并（merge）与变基（rebase）

两者都是“把 A 分支的改动接到 B 分支上”，但方式不同：

- **merge**: 把两条线的历史“缝合”，保留双方各自的提交，产生一个合并提交。适合“你改了 10 个提交、上游改了 50 个提交”的长周期场景。本指南默认用它。
- **rebase**: 把你的提交“抽出来、在最新上游之上重新逐条重放”，历史变成一条直线。历史更干净，但遇到冲突时要逐提交解决，复杂度高。

```
# 在开发分支上合并最新官方代码：
git switch my-dev
git fetch upstream
git merge upstream/master
# 或（想保持线性历史时）：
git rebase upstream/master
```

推荐：先以 merge 为主，简单可靠；熟练后再考虑 rebase。

5 两条二次开发路线

这是全文档最重要的决策。两条路线对“升级同步”的友好度差别巨大。

5.1 路线 A：直接修改核心包（不推荐用于长期定制）

直接改 packages/ 下某个包的源码，例如给 packages/core 的 agent-loop 加行为、改 packages/llm 的提示词。

- **优点**：能改一切，包括官方未开放的内部行为；不需要学习插件接口。
- **缺点**：官方 pre-release 阶段频繁重构、重命名、重打包，你改的同一段代码极可能被官方改动覆盖，升级时冲突最多、最难自动迁移。仓库纪律明确要求“改 agent-loop 必须同步更新架构文档”，说明这类改动是最深层的耦合。

5.2 路线 B：写插件（推荐）

“万物皆插件”——把定制能力做成一个插件，通过 cordis.yml 挂载，不触碰官方包源码。这也是官方文档教程（docs/user/develop/basic/）的第一课。

一个最简单的插件（hello-plugin.ts）：

```
import type { Context } from '@deepseek-ai/cordis'

export const name = 'hello-plugin'

export function apply(ctx: Context) {
  console.log('[hello-plugin] plugin loaded!')
}
```

在项目根建一个 scratch 目录，用 --patch 挂载：

```
mkdir -p scratch-plugin/src
# 把上面的文件存为 scratch-plugin/src/my-plugin.ts
# 建 scratch-plugin/cordis.yml，内容见下，然后：
pnpm dsh web --patch ./scratch-plugin/cordis.yml

- insert:
  - id: hello
    name: '/绝对路径/deepseek-harness/scratch-plugin/src/my-plugin.ts'
```

插件可以注册的能力包括：工具（ctx.tools）、钩子（ctx.on 监听扩展点，例如权限门 tools/pre-execute）、事件监听、服务（继承 Service）、UI 插件（监听 session/event）等。详见 docs/cookbook/extension-cookbook.md 与 docs/user/develop/。

5.3 路线对比

维度	路线 A：改核心包	路线 B：写插件
可定制范围	一切（含内部行为）	官方开放的扩展点、事件、服务
学习成本	低（直接改）	中（要学 Cordis 插件模型）
与官方耦合	极高	低
官方升级冲突	频繁、难自动	几乎无冲突，插件代码基本不动
Vibe coding 迁移难度	高（每次要重做）	低（一次做好，长期复用）
适用场景	临时验证、官方没有扩展点	长期二次开发

建议：长期二次开发以路线 B 为主。确需修改核心行为时，把改动记录成“一份补丁清单”，便于升级时重放（见第 6 节）。

6 升级同步流程（核心）

6.1 策略总览

- 官方仓库（upstream/master）是“上游”，持续向前演进。
- 你的所有定制都存在于自己的开发分支（推荐路线 B 的插件代码，连同你自己的 cordis.yml），并 push 到 origin（你的 Fork）做云端备份。
- 官方发新版时，把它 merge 进你的开发分支；若路线 B，你的插件代码几乎不动，只有少量接口签名变化需要适配。
- 合并结果 push 回 origin，云端 Fork 也保持与官方同步。
- 冲突解决、接口适配交给 vibe coding（AI）辅助完成。

6.2 升级操作的详细步骤

1. 提交或暂存当前未提交的改动，保证工作区干净：

```
git status          # 确认没有未提交改动
git stash           # 若有，先暂存起来；稍后 git stash pop 取回
```

2. 拉取官方上游最新：

```
git fetch upstream
git log --oneline master..upstream/master # 先看看官方新增了什么
```

3. 切到你的开发分支并合并官方上游：

```
git switch my-dev
git merge upstream/master
```

4. 若无冲突：git push 把合并结果推回你的 Fork，升级完成。若报 CONFLICT，进入第 6.3 节。

附带收益：你的 Fork 只是你的私人副本，push 永远允许、不会污染官方仓库；若你想把定制贡献回官方，再在 GitHub 上对你的 Fork 提 Pull Request（合并 upstream 后 PR 更易被官方接受）。

6.3 冲突是什么、怎么处理

冲突 = 你和上游都改了同一文件同一处，git 无法自动决定以谁为准。冲突只影响你同时改过的地方，不影响其余代码。

1. 查看冲突文件：

```
git status          # “both modified” 的就是冲突文件
```

2. 打开冲突文件，找 <<<<<<、=====、>>>>>> 标记，手动取舍两版内容，删掉标记行。

3. 标记为已解决并提交（merge 时）：

```
git add <已解决的文件>
git commit          # 完成合并提交；或 git merge --continue
```

若冲突太多、局面失控：git merge --abort 放弃本次合并，回到合并前状态，另想办法（例如先交给 AI 分步处理）。

6.4 把升级与冲突交给 vibe coding

当冲突较多、或官方 API 变了需要适配插件时，把工作交给 AI 完成，流程建议如下：

1. **准备：**工作区干净，开发分支存在，冲突已由 git 检出到文件里（或刚拉好上游、尚未开始合并）。
2. **给 AI 一份清晰的指令：**包括“仓库路径、目标（合并 upstream/master 到 my-dev）、约束（只处理冲突/适配，不改无关代码）、验证命令（`pnpm run typecheck`、`pnpm run build`、`pnpm run test`）”。
3. **让 AI 处理：**它会读 AGENTS.md 了解仓库纪律，逐文件解决冲突，并运行验证命令。
4. **人工复核：**查看 AI 的改动（`git diff`），重点确认它没有顺手改了不该改的东西；跑一遍 `pnpm dsh web` 做冒烟测试。
5. **提交推送：**`git add -A && git commit -m "merge upstream rc.5 -> rc.6"`。

6.5 写给 AI 的提示词模板

我正在对 DeepSeek Harness 做二次开发，仓库位于 <repo路径>。
当前分支 <my-dev>，刚刚执行了 `git merge upstream/master`，产生了如下冲突：
<粘贴 `git status` 里标为 both modified 的文件列表>

请完成：

1. 先阅读仓库根目录 AGENTS.md，遵守其纪律。
2. 只处理本次合并的冲突，以及因上游 API 变更导致我自研插件无法编译的适配；不要改动任何与升级无关的代码。
3. 解决冲突时，参考 <<<<<< ===== >>>>>> 两边的意图：上游新逻辑保留，我的定制逻辑保留并适配到新接口。
4. 完成后依次运行并保证通过：`pnpm run typecheck`、`pnpm run build`；若相关测试可跑，运行 `pnpm run test` 的相关用例。
5. 最后用不超过 10 行说明：改了哪些文件、每个冲突如何取舍、还有哪些未完成项。

6.6 长期维护建议

- **定期升级：**不要等官方攒了几十个版本再合，建议每次官方发布就 `merge upstream/master` 一次，冲突面更小。
- **定期推送备份：**开发分支改动后及时 `push` 到 origin（你的 Fork），本地损坏也不丢。
- **插件代码独立成仓库：**把自研插件放进单独的 git 仓库（不混在官方仓库里），官方仓库升级时甚至可以根本不拉，只在需要时重新对官方 API 做一次适配。
- **用补丁清单记账：**若确需改核心包，维护一份 CHANGES.md，逐条记录“改了哪个文件、为什么、如何适配新版本”，交给 AI 做迁移时它就是说明书。

- **保持工作区干净：**任何升级前先 commit 或 stash，给 AI 一个干净的起点，也便于失败后回滚（git reflog 可找回被 reset 的提交）。

7 故障排查（实战记录）

本节记录本机在搭建环境、安装插件、修复皮肤时遇到的真实故障与排查过程。每一条都按「现象 → 排查 → 根因 → 修复」记录，遇到类似问题可直接对照。

7.1 双击启动脚本闪退

- **现象：**双击 启动-DeepSeek-Harness.bat，窗口一闪而过，报错内容看不清。
- **排查：**把输出重定向到日志文件（pnpm dsh web > web.log 2>&1）逐行看。先后发现两类根因。
- **根因一（pnpm 版本错配）：**.bat 曾用全局 pnpm（本机 v10.33.0），而仓库 package.json 钉死 pnpm@11.7.0。pnpm v10 在 v11 workspace 里跑 pnpm dsh web 时，源码启动命令 node --import tsx/esm apps/cli/src/bin.ts 的相对路径解析异常，跑到另一个仓库（master），而 master 从未 pnpm run build（无 lib/ 产物），加载插件必然报错退出。修复：.bat 优先用 corepack pnpm。
- **根因二（端口被占用）：**若 3080 已被一个正在运行的 dsh 实例占用，再次双击启动第二个实例会报 Error: listen EADDRINUSE: address already in use 127.0.0.1:3080。修复：.bat 启动前先 netstat 检测端口，已占用则提示「服务已在运行」并直接打开浏览器，不重复启动。
- **教训：**启动 dsh 必须用仓库匹配的 pnpm 版本，且避免多实例端口冲突。判断方法：启动后看日志里 node ... bin.ts "web" 的完整路径，确认是当前仓库；端口被占时看 netstat -ano | findstr :3080。

7.2 Web 启动报 plugin tree failed to load

- **现象：**启动后报 Error: dsh: plugin tree failed to load: loader fibers failed, 附带 Cannot find module '...\@deepseek-ai\dsh-settings\lib\index.js'。
- **根因：**\$DSH_HOME/profiles/node_modules(~.dsh/profiles/node_modules)里的 @deepseek-ai/* 符号链接被错误地指向了 master 仓库。这个目录是 dsh 启动时由 healProfilesModuleFallback 自动维护的「依赖闭包扁平链接」，指向 ** 本次运行 dsh 所在安装 ** 的依赖。若 dsh 曾在 master 仓库启动过，链接就指向 master；master 没构建（无 lib/），插件解析 @deepseek-ai 依赖就失败。

- **排查：** `ls -la ~/.dsh/profiles/node_modules/@deepseek-ai/` 看链接目标是否指向错误的仓库。
- **修复：** 从正确的仓库（penguin）重新启动 dsh，启动时自动重建这些链接。确认方法：`readlink ~/.dsh/profiles/node_modules/@deepseek-ai/dsh-settings` 应指向 penguin 的路径。
- **教训：** 不要从多个仓库副本交叉启动 dsh。同一份 `~/.dsh` 配置只配对一个仓库安装；切换仓库后，先确认 profile 链接重建成功再启动。

7.3 皮肤中心「应用失败 ENOENT scandir node_modules/skins/」

- **现象：** 在皮肤中心点任意皮肤，报 应用失败（ENOENT: no such file or directory, scandir '...\profiles\web\node_modules\skins\'）。
- **根因：** 这是 dsh-web-ui 插件在 npm 安装下的已知 bug（上游 issue #21）。插件 host 端硬编码用 `../../../skins/` 相对路径解析皮肤目录，源码仓库布局下解析到 `packages/skins/`，npm 布局下解析成 `node_modules/skins/`——该目录不存在。且插件假设皮肤包是符号链接，而 pnpm 安装的是真实目录，`ensureSymlink` 会拒绝。
- **修复（改插件产物，不动 DSH 核心）：**
 1. 在 `profiles/web/node_modules/skins/` 为皮肤包建立目录联接（junction），指向 `node_modules/@linxin666` 已装 7 款：`skins/xp`、`skins/thx`、`skins/blue-fantasy`、`skins/qq98`、`skins/dragon-heir`、`skins/minecraft`、`skins/whale-song`（鲸吟需先单独安装，见第 7.5 节）。
 2. 补丁 `@linxin666/dsh-client-ui-skin-center/lib/index.js` 两处：`ensureSymlink` 中目标存在（真实目录或符号链接）时跳过建链直接返回；`checkInstalled` 改为 `isSymbolicLink() || isDirectory()` 都算已安装。
- **注意：** 重装 `dsh-web-ui-all` 会覆盖补丁、清掉 junction，需重做。等上游发布修复版后重装即可。
- **副作用：** 切换后 `~/.dsh/cordis.patch.yml` 会写皮肤 managed 区段，其中对「聚合包未挂载的皮肤行」的 `disabled` 会打印 `entry not found` 软警告，不影响功能。

7.4 技术文档位置错乱

- **现象：** 二次开发指南.tex/pdf 出现在 master（旧 ZIP 快照仓库）里，而 penguin（fork 且有 git）才是实际开发仓库。
- **根因：** 早期在 master 快照上建文档，后来切换到 penguin 仓库开发，文档没有及时迁移。
- **修复：** 文档统一放在实际开发仓库（penguin），删除 master 里的残留。
- **教训：** 一个开发仓库只配一份文档与配置文件。master 是无 git 的 ZIP 快照，不要在里面放正式文件；一切改动只发生在 fork 仓库（penguin）。

7.5 皮肤缺款：鲸吟（whale-song）无法切换

- 现象：皮肤中心/接口报 `unknown skin "whale-song". Known: blue-fantasy, dragon-heir, minecraft, qq98, ths, xp`。
- 根因：dsh-web-ui-all@0.1.2 聚合包的依赖只带了 6 款皮肤，鲸吟（whale-song）虽已发布到 npm（0.1.2）但未随聚合包分发，也未在 `node_modules/skins/` 注册。
- 修复：
 1. 用与 profile store 匹配的 pnpm 版本把 @linxin666/dsh-client-ui-skin-whale-song 装进 profile（见下一条的 store 版本问题）；
 2. 在 `profiles/web/node_modules/skins/` 建 whale-song 目录联接，指向 `node_modules/@linxin666/dsh-client-ui-skin-whale-song`；
 3. 重启 web，皮肤中心即识别。

7.6 外部皮肤：深海女仆工坊（maid-atelier）

社区独立分发的鲸鱼娘皮肤（github.com/Small-tailqwq/dsh-deep-whale，CC BY-NC-SA 4.0 非商用），也是标准皮肤包格式，可与皮肤中心共存：

- 安装：clone 仓库到本地，用与 profile store 匹配的 pnpm 版本把皮肤包加进 profile，然后在 `node_modules/@dsh-external/` 与 `node_modules/skins/` 各建一个指向真实皮肤目录的 junction（D:\...\dsh-deep-whale\maid-atelier）。注意：pnpm 的 `link:` 方式在 hoisted 布局下不稳定（会被后续 install 清掉），用 junction 直连真实路径最稳。
- 验证：重启 web 后皮肤中心应出现 maid-atelier（深海女仆工坊），可自由切换。
- 许可：该皮肤为衍生创作，CC BY-NC-SA 4.0，禁止商用；署名链见其 NOTICE。

7.7 pnpm store 版本冲突

- 现象：用 `dsh plugin --profile web add < 包 >` 装包时报 `pnpm now wants to use the store at ...store\v11 ... reinstall your dependencies`。
- 根因：profile 的 `node_modules` 是用 pnpm v10 装的（`.modules.yaml` 记录 `packageManager: pnpm@10.33.0`、`storeDir` 指向 `store/v10`），而 dsh plugin 用 v11 执行，store 目录不匹配。
- 修复：用与 profile 现有 `.modules.yaml` 一致的 pnpm 版本手动装。例如在 profile 目录用全局 v10 pnpm：`"%APPDATA%\npm\pnpm.cmd" add < 包名 >`。
- 注意：不要轻易删 profile 的 `node_modules` 全量重建——会牵动 `profiles/node_modules` 的 @deepseek-ai 链接归属（见第 7.2 节），代价高。优先保持单一 pnpm 版本维护 profile。

7.8 故障排查通用流程

1. **先抓住完整报错**：窗口一闪而过时，用命令把输出重定向到文件再看，别靠肉眼。例如 `pnpm dsh web > web.log 2>&1`。
2. **看完整错误栈的路径**：报错里的每个路径都值得读——它揭示了 `dsh` 实际在哪个仓库、解析到哪个包的哪个文件。
3. **确认 pnpm 版本**：`pnpm --version` 是否等于仓库 `package.json` 的 `packageManager`。不一致时用 `corepack pnpm`。
4. **确认 profile 链接归属**：`ls -la ~/.dsh/profiles/node_modules/@deepseek-ai/`，看是否指向当前仓库。
5. **一次只从当前仓库启动**：避免多仓库交叉污染 `~/.dsh` 配置。

8 常见问题

问 没有 `.git` 除了不能更新 GitHub 文件，还有别的坏处吗？

答 是的。没有 `git` 仓库就无法：管理自己的改动历史（diff/回滚/分支）、对比与官方版本的差异、用 `git` 工具辅助升级、把开发分支推送到远程备份。本章第 6 节的所有同步操作全部依赖 `git`。

问 Fork 之后官方更新，我账号里的 Fork 会自动同步吗？

答 不会。Fork 是点击那一刻的快照，官方再更新不会自动过来。要同步：GitHub 网页点 **Sync fork** 按钮，或本地执行 `git fetch upstream && git merge upstream/master` 后再 `git push`。升级流程见第 6 节。

问 官方升级后我的二次开发会“打水漂”吗？

答 只要你的改动被 `git commit` 了，就永远在历史里，不会丢。会被“冲掉”的只是冲突时你未选择保留的那一行代码。路线 B 下，插件代码在官方升级时几乎不动，迁移成本最低。

问 必须跟着官方最新版本走吗？

答 不必。rc 版官方不承诺兼容，你完全可以钉在一个稳定版本上开发。跟随与否取决于你是否需要官方新功能/修复。策略见第 6 节。

问 `pnpm run typecheck` 必须要过吗？

答 这是官方定义的“安装完成”标准（`docs/development.md`），也是升级后验证改动没有破坏类型契约的最低门槛，建议始终以它作为完成基准。

问 我在 Windows 上，会有什么坑？

答 本仓库 Windows 有专门的 gate (`pnpm run check:windows-wine`) 由 CI 负责；本机开发时建议用 bash 风格终端（如 Git Bash）执行上面的命令，避免 `.cmd` 路径问题。冲突标记文件编辑用支持 UTF-8 的编辑器即可。

附：参考文档

- 仓库自述：README.md、README.zh.md
- 开发指南：docs/development.md
- 架构与扩展点：docs/architecture.md
- 插件教程：docs/user/develop/basic/index.md（首个插件）、publish.md（打包安装）
- 插件形态参考：docs/cookbook/extension-cookbook.md
- 新增包清单：docs/cookbook/adding-a-package.md
- 仓库纪律（供 AI 阅读）：根目录 AGENTS.md